RK805 Developer Guide

ID: RK-KF-YF-127

Release Version: V1.1.2

Release Date: 2026-06-24

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, No.89, Software Boulevard, TongPan Rd, Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document primarily introduces the various sub-modules of the RK805, covering related concepts, functionalities, DTS configuration, and analysis and troubleshooting of common issues.

Product Version

Chip Models	Kernel Version
RK805	3.10, 4.4, 4.19

Intended Audience

This document (this guide) is primarily intended for the following engineers:

- Technical Support Engineers
- Software Development Engineers

Revision History

Version	Author	Date	Modification Description
V1.0.0	Chen Jianhong	2017-05-28	Initial version
V1.1.0	Chen Jianhong	2019-11-11	Added support for kernel 4.19
V1.1.1	Huang Ying	2022-05-30	Format modification
V1.1.2	Chen Jianhong	2026-06-24	Fixed some typos

Table of Contents

RK805 Developer Guide
1. Basics
1.1 Overview
1.2 Functions
1.3 Chip Pin Functions
1.4 Key Concepts
1.5 Power-On Conditions and Sequence
2. Configuration
2.1 Driver and menuconfig
2.2 DTS Configuration
2.3 Function Interfaces
3. Debug
3.1 Kernel 3.10
3.2 Kernel 4.4
3.3 Kernel 4.19

1. Basics

1.1 Overview

The RK805 is a high-performance PMIC integrating four high-current DCDC converters, three LDOs, one RTC, and adjustable power-on sequencing functions.

Power rails in the system are generally categorized into two types: DCDC and LDO. The general characteristics of these two power types are as follows (please search for detailed information independently):

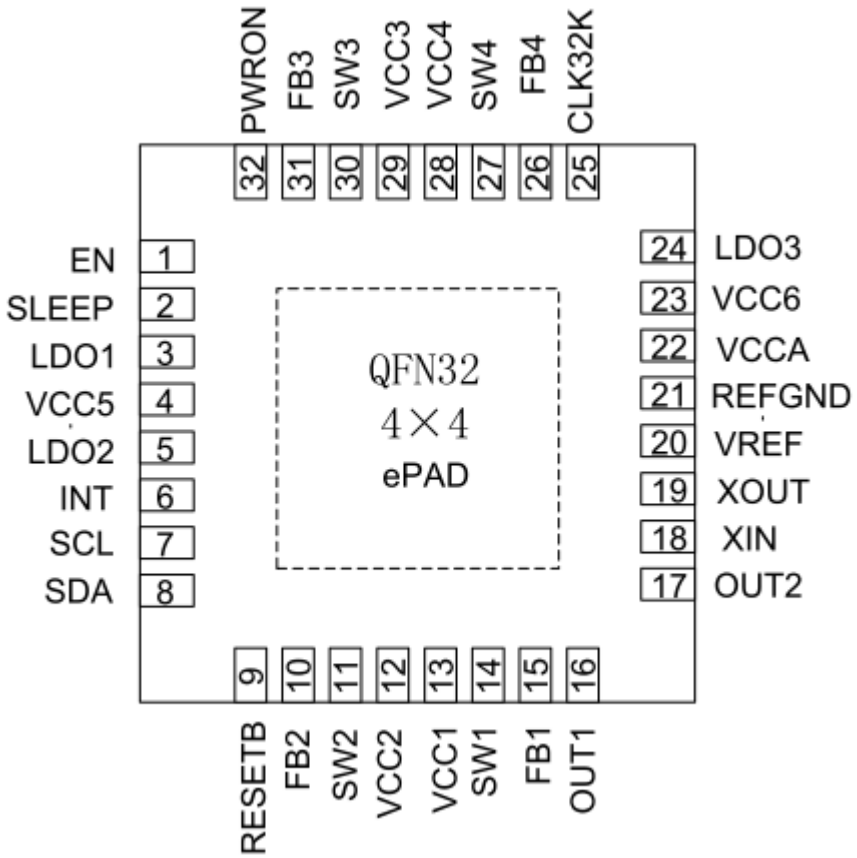
1. DCDC: Offers high efficiency when the input-output voltage difference is large, but suffers from relatively high ripple and higher cost. Therefore, it is used for applications with large voltage differences and high current loads. It generally operates in two modes. PWM mode: Good ripple and transient response, but lower efficiency; PFM mode: High efficiency, but poor load capability.
2. LDO: Offers low efficiency and low cost when the input-output voltage difference is large. To improve LDO conversion efficiency, the system implements relevant optimizations. For example, if the LDO output voltage is 1.1V, its input voltage can be supplied by the VCCIO_3.3V DCDC to improve efficiency. Therefore, if the circuit allows, LDOs should be connected to the DCDC output circuit as much as possible, but power-on sequencing must be considered.

1.2 Functions

From the user's perspective, the functions of the RK805 can be summarized into four parts:

1. Regulator function: Controls the status of various DCDC and LDO power rails;
2. RTC function: Provides clock timing, alarm, and other functions;
3. GPIO function: Two push-pull output pins, OUT1 and OUT2 (output only), which can be used as output-only GPIOs;
4. PWRKEY function: Detects power button press/release events, saving one GPIO for the AP.

1.3 Chip Pin Functions



In the following description, special attention should be paid to the SLEEP and INT pins:

PIN NO	PIN NAME	PIN DESCRIPTION
1	EN	Power on or power off enable pin, active high, internal 800k resistor pull low to ground
2	SLEEP	Sleep mode control input
3	LDO1	LDO1 output
4	VCC5	Power supply of LDO1/2
5	LDO2	LDO2 output
6	INT	Interrupt request pin, open drain
7	SCL	I2C clock input
8	SDA	I2C data input and output
9	RESETB	Reset pin after power on, active low
10	FB2	Output feedback voltage of buck2
11	SW2	Switching node of buck2
12	VCC2	Power supply of buck2
13	VCC1	Power supply of buck1
14	SW1	Switching node of buck1
15	FB1	Output feedback voltage of buck1
16	OUT1	General digital output pin 1, CMOS level output, high level is VFB4
17	OUT2	General digital output pin 2, CMOS level output, high level is VFB4
18	XIN	32.768KHz crystal oscillator input
19	XOUT	32.768KHz crystal oscillator output
20	VREF	Internal reference voltage
21	REFGND	Reference ground
22	VCCA	Power supply of controller
23	VCC6	Power supply of LDO3

PIN NO	PIN NAME	PIN DESCRIPTION
24	LDO3	LDO3 output
25	CLK32K	32.768KHz clock output, open drain
26	FB4	Output feedback voltage of buck4
27	SW4	Switching node of buck4
28	VCC4	Power supply of buck4
29	VCC3	Power supply of buck3
30	SW3	Switching node of buck3
31	FB3	Output feedback voltage of buck3
32	PWRON	Power on key input, active low, internal 17k resistor pull high to VCCA
Exposed pad	Exposed ground	Ground

1.4 Key Concepts

- I2C Address

7-bit slave address: 0x18

- The PMIC has three operating modes

1. PMIC Normal Mode

When the system is running normally, the PMIC operates in normal mode, with the pmic_sleep pin held low.

2. PMIC Sleep Mode

To minimize standby power consumption during system suspend, the PMIC switches to sleep mode. In this mode, output voltages for certain rails may be reduced or outputs disabled entirely, configurable based on specific product requirements. During system standby, the AP configures the pmic_sleep mode via I2C commands and then drives the pmic_sleep pin high to enter sleep state. Upon SoC wakeup, the pmic_sleep pin returns to low, causing the PMIC to exit sleep mode.

3. PMIC Shutdown Mode

During system shutdown, the PMIC executes power-down sequences for the entire system. The AP configures the pmic_sleep mode to shutdown via I2C commands and then drives the pmic_sleep pin high to enter shutdown state.

- pmic_sleep Pin

Normally held low, indicating PMIC normal mode. When driven high, the PMIC switches to either sleep or shutdown mode.

- pmic_int Pin

Normally held high. When an Interrupt occurs, the pin transitions to low. If the Interrupt remains unhandled, the pin stays low.

- out1/out2 Pins

These pins can function as general-purpose GPIOs (push-pull output), supporting only GPIO output mode.

- pmic_pwron Pin

The pwrkey function requires hardware connection of the power button to this pin. The Driver uses this pin to detect button press and release events.

- Operating Modes of Individual DCDC Rails

  The DCDC supports PWM (also known as force PWM) and PFM modes. However, the PMIC features a mode that dynamically switches between PWM and PFM, commonly referred to as AUTO mode. The PMIC supports two configurations: PWM mode and AUTO PWM/PFM mode. While AUTO mode offers higher efficiency, it exhibits poorer ripple and transient response. For system stability, the device is configured to operate in PWM mode during runtime, and switches to AUTO PWM/PFM mode when the system enters sleep state.

- DCDC3 Voltage Regulation

The DCDC3 power rail is unique; its voltage cannot be modified via registers and must be adjusted using external voltage divider resistors. Therefore, to change the voltage, the peripheral hardware must be modified. In Rockchip solutions, this rail is typically used as VCC_DDR.

- Runtime Voltage Adjustment Range for DCDC and LDO

1. Discontinuous DCDC Voltage Range:

Voltage Range (V)	Step Value (mV)	Specific Voltage Levels (V)
0.7125 ~ 1.45	12.5	0.7125, 0.725, 0.7375, ..., 1.45
1.8 ~ 2.2	200	1.8, 2.0, 2.2
2.3	N/A	2.3

2. Continuous LDO Voltage Range:

Voltage Range (V)	Step Value (mV)	Specific Voltage Levels (V)
0.8 ~ 3.4	100	0.8, 0.9, 1.0, 1.1, 1.2, ..., 3.4

1.5 Power-On Conditions and Sequence

1. Power-On Conditions

The PMIC can be powered on if any of the following conditions are met:

- The EN signal transitions from low to high.
- The EN signal remains high, and an RTC alarm Interrupt is triggered.
- The EN signal remains high, and the PWRON button is pressed.

2. Power-On Sequence

Power-on sequence requirements for each power rail may vary depending on the SoC platform. The current power-on sequences are listed below; please refer to the latest datasheet for specific details:

AP			Null		RK3228		RK1108	
BOOT(OTP)			0		1			
			RK805-0		RK805-1		RK805-2	
	Output voltage range	Max output current	Default voltage	Power sequence	Default voltage	Power sequence	Default voltage	Power sequence
BUCK1	0.7125V-1.45V(step 12.5mV) /1.8V/2V/2.2V/2.3V	2.5A	1.0V	2	1.1V	2	1.0V	2
BUCK2	0.7125V-1.45V(step 12.5mV) /1.8V/2V/2.2V/2.3V	2.5A	1.0V	2	1.1V	2	1.0V	X
BUCK3	setting by external resistors	1.5A	X	3	X	3	X	3
BUCK4	0.8V-3.5V(step=0.1V)	1.5A	3.3V	5	3.3V	5	3.3V	5
LDO1	0.8V-3.4V(step=0.1V)	300mA	1.0V	1	1.8V	4	1.0V	1
LDO2	0.8V-3.4V(step=0.1V)	300mA	1.8V	4	1.8V	4	1.8V	4
LDO3	0.8V-3.4V(step=0.1V)	100mA	1.0V	1	1.0V	1	1.0V	1
RESETB			X	8	X	10	X	10

Table 4-1 Power Start Up Sequence

2. Configuration

2.1 Driver and menuconfig

Kernel 3.10 Configuration

RK805 driver file (reuses RK816 driver):

```
drivers/mfd/rk816.c
drivers/input/misc/rk816-pwrkey.c
drivers/rtc/rtc-rk816.c
drivers/gpio/gpio-rk816.c
drivers/regulator/rk816-regulator.c
```

Corresponding macro configuration in menuconfig:

```
CONFIG_MFD_RK816
CONFIG_GPIO_RK816
CONFIG_RTC_RK816
CONFIG_REGULATOR_RK816
CONFIG_INPUT_RK816_PWRKEY
```

Kernel 4.4 Configuration

RK805 Driver file:

```
drivers/mfd/rk808.c
drivers/input/misc/rk8xx-pwrkey.c
drivers/rtc/rtc-rk808.c
drivers/gpio/gpio-rk8xx.c
drivers/regulator/rk818-regulator.c
drivers/clk/clk-rk808.c
```

Corresponding macro configuration in menuconfig:

```
CONFIG_MFD_RK808
CONFIG_RTC_RK808
CONFIG_GPIO_RK8XX
CONFIG_REGULATOR_RK818
CONFIG_INPUT_RK8XX_PWRKEY
CONFIG_COMMON_CLK_RK808
```

Kernel 4.19 Configuration

RK805 Driver file:

```
drivers/mfd/rk808.c
drivers/input/misc/rk805-pwrkey.c       // Different from the Kernel 4.4
drivers/rtc/rtc-rk808.c
drivers/pinctrl/pinctrl-rk805.c         // Different from the Kernel 4.4
drivers/regulator/rk808-regulator.c     // Different from the Kernel 4.4
drivers/clk/clk-rk808.c
```

Corresponding macro configuration in menuconfig:

```
CONFIG_MFD_RK808
CONFIG_RTC_RK808
CONFIG_PINCTRL_RK805
CONFIG_REGULATOR_RK808
CONFIG_INPUT_RK805_PWRKEY
CONFIG_COMMON_CLK_RK808
```

2.2 DTS Configuration

3.10 Kernel Configuration

DTS configuration includes: I2C bus attachment, main node, regulator, RTC, poweroff, and other sections.

```
&i2c1 {
    rk805: rk805@18 {
        reg = <0x18>;
        status = "okay";
    };
};
```

```
#include "./../arm/boot/dts/rk805.dtsi"
&rk805 {
    gpios = <&gpio2 GPIO_A6 GPIO_ACTIVE_HIGH>, <&gpio2 GPIO_D2 GPIO_ACTIVE_LOW>;
    rk805,system-power-controller;
    gpio-controller;
    #gpio-cells = <2>;

    rtc {
        status = "disabled";
    };

    regulators {
        rk805_dcdc1_reg: regulator@0 {
            regulator-name = "vdd_logic";
            regulator-min-microvolt = <712500>;
            regulator-max-microvolt = <1450000>;
            regulator-initial-mode = <0x1>;
            regulator-initial-state = <3>;
            regulator-boot-on;
            regulator-always-on;
            regulator-state-mem {
                regulator-state-mode = <0x2>;
                regulator-state-enabled;
                regulator-state-uv = <1000000>;
            };
        };
        rk805_dcdc2_reg: regulator@1 {
                    ............
        };
        rk805_dcdc3_reg: regulator@2 {
                    ............
        };
        ..............................
    };
};
```

1. I2C Attachment

The complete rk805 node must be attached under the corresponding I2C node, with `status = "okay";` configured.

2. Main Section

- Non-modifiable parts

```
    rk805,system-power-controller: Declare that the RK805 supports system power-down
management.
    gpio-controller: Declares that the RK805 has GPIO functionality.
    #gpio-cells: The number of parameters that must be specified when referencing the RK805
GPIO.
```

Note: If a node needs to reference the RK805 GPIO, use the following format:

`gpios = <&rk805 0 GPIO_ACTIVE_LOW>;` First parameter: &rk805 is fixed and must not be changed; Second parameter: Specifies which RK805 GPIO to reference; valid values are 0 or 1, where 0 corresponds to out1 and 1 corresponds to out2; Third parameter: GPIO polarity.

- Modifiable sections

gpios: Specifies the pmic_int (first) and pmic_sleep (second) pins;

3. Regulator section

- `regulator-name`: Power supply name; it is recommended to keep this consistent with the hardware schematic, as this name must match when using the regulator_get interface;
- `regulator-min-microvolt`: Minimum voltage adjustable at runtime;
- `regulator-max-microvolt`: Maximum voltage adjustable at runtime;
- `regulator-initial-mode`: DCDC operating mode at runtime; generally configured as 1. 1: force pwm, 2: auto pwm/pfm;
- `regulator-state-mode`: DCDC operating mode during sleep; generally configured as 2. 1: force pwm, 2: auto pwm/pfm;
- `regulator-initial-state`: Mode during suspend; must be configured as 3;
- `regulator-boot-on`: If this property exists, the power supply will be enabled upon regulator registration;
- `regulator-always-on`: If this property exists, the power supply cannot be disabled at runtime and will be enabled upon registration;
- `regulator-state-enabled`: Maintains power state during sleep; to disable this power supply, change to "regulator-state-disabled";
- `regulator-state-uv`: Standby voltage when power is maintained during sleep.

Note:

If the values of regulator-min-microvolt and regulator-max-microvolt are equal, the system framework will automatically set this voltage and enable the power rail during regulator registration, without requiring user intervention.

If `regulator-boot-on` or `regulator-always-on` is present, the system framework will enable the regulator by default during registration. In this case, the regulator voltage behaves as follows: if `regulator-min-microvolt` and `regulator-max-microvolt` are equal, the system framework sets the voltage to this specific value; if they differ, the voltage defaults to the PMIC's hardware power-on default voltage.

4. RTC Section

If RTC functionality is not required (e.g., for box products), add the node as described above and explicitly set `status = "disabled"`. To enable RTC, either remove the entire RTC node or set its status to `status = "okay"`.

5. Poweroff Section

```
gpio_poweroff {
        compatible = "gpio-poweroff";
        gpios = <&gpio2 GPIO_D2 GPIO_ACTIVE_HIGH>;
        status = "okay";
};
```

Since the RK805 supports powering down the entire PMIC by pulling the pmic_sleep pin high, this node must be added under the root node. The gpios property is configurable and specifies the pmic_sleep pin.

Kernel 4.4 Configuration

The DTS configuration includes: I2C bus attachment, main device, RTC, power key, GPIO, regulator, and other components.

```
&pinctrl {
        pmic {
                pmic_int_l: pmic-int-l {
                rockchip,pins =
                        <2 6 RK_FUNC_GPIO &pcfg_pull_up>;       /* gpio2_a6 */
                };
        };
};

&i2c1 {
        status = "okay";
        rk805: rk805@18 {
                compatible = "rockchip,rk805";
                status = "okay";
                reg = <0x18>;
                interrupt-parent = <&gpio2>;
                interrupts = <6 IRQ_TYPE_LEVEL_LOW>;
                pinctrl-names = "default";
                pinctrl-0 = <&pmic_int_l>;
                rockchip,system-power-controller;
                wakeup-source;
                gpio-controller;
                #gpio-cells = <2>;
                rtc {
                        status = "disabled";
                };
                pwrkey {
                        status = "disabled";
                };
                gpio {
                        status = "okay";
                };
                regulators {
                        compatible = "rk805-regulator";
                        status = "okay";
                        #address-cells = <1>;
                        #size-cells = <0>;
                        vdd_logic: RK805_DCDC1@0 {
                                regulator-compatible = "RK805_DCDC1";
                                regulator-name = "vdd_logic";
                                regulator-init-microvolt = <1100000>;
                                regulator-min-microvolt = <712500>;
                                regulator-max-microvolt = <1450000>;
                                regulator-initial-mode = <0x1>;
                                regulator-ramp-delay = <12500>;
```

```
                                regulator-boot-on;
                                regulator-always-on;
                                regulator-state-mem {
                                        regulator-mode = <0x2>;
                                        regulator-on-in-suspend;
                                        regulator-suspend-microvolt = <1000000>;
                                };
                        };

                        vdd_arm: RK805_DCDC2@1 {
                                ..................
                        };
                        vcc_ddr: RK805_DCDC3@2 {
                                ..................
                        };
                        ..............................
                };
        };
};
```

1. I2C Attachment

The complete RK805 node must be attached under the corresponding I2C node, with `status = "okay";` configured.

2. Main Section

- Do not modify:

```
compatible = "rockchip,rk805";
reg = <0x18>;
rockchip,system-power-controller;
wakeup-source;
gpio-controller;
#gpio-cells = <2>;
```

- Modifiable (according to pinctrl rules)

interrupt-parent: Specifies which GPIO the pmic_int belongs to; interrupts: Specifies the pin index and polarity of pmic_int on the GPIO defined in interrupt-parent; pinctrl-names: Do not modify; fixed as "default"; pinctrl-0: References the pmic_int pin defined in pinctrl;

3. rtc, pwrkey, gpio

If these modules are selected in menuconfig but the corresponding Drivers do not need to be enabled in practice, you can add the rtc, pwrkey, and gpio nodes in the DTS and explicitly set their status to status = "disabled". This prevents the Drivers from being enabled, although error logs will appear during boot and can be ignored. To enable the Drivers, either remove the corresponding nodes or set their status to status = "okay".

4. regulator

- `regulator-compatible`: The name used for matching during Driver registration. Do not modify this value, or the Driver will fail to load.

- `regulator-name`: The power supply name. It is recommended to keep this consistent with the hardware schematic, as the `regulator_get` interface requires a match with this name.
- `regulator-init-microvolt`: The initialization voltage during the U-Boot stage; this is invalid in the kernel stage.
- `regulator-min-microvolt`: The minimum voltage adjustable during runtime.
- `regulator-max-microvolt`: The maximum voltage adjustable during runtime.
- `regulator-initial-mode`: The DCDC operating mode during runtime, typically configured as 1. Values: 1 (force PWM), 2 (auto PWM/PFM).
- `regulator-mode`: The DCDC operating mode during sleep, typically configured as 2. Values: 1 (force PWM), 2 (auto PWM/PFM).
- `regulator-initial-state`: The mode during suspend; must be configured as 3.
- `regulator-boot-on`: When this property is present, the power supply is enabled upon Driver registration.
- `regulator-always-on`: When this property is present, the power supply cannot be disabled during runtime and is enabled upon Driver registration.
- `regulator-ramp-delay`: The DCDC voltage rise time; fixed at 12500.
- `regulator-on-in-suspend`: Maintains power during sleep. To disable the power supply during sleep, change this to `regulator-off-in-suspend`.
- `regulator-suspend-microvolt`: The standby voltage when power is maintained during sleep.

Kernel 4.19 Configuration

Refer to the Kernel 4.4 DTS configuration. Difference: The Kernel 4.19 DTS configuration no longer requires the gpio subnode, but other modules still reference and use RK805 pins using the method `gpios = <&rk805 0 GPIO_ACTIVE_LOW>;`.

2.3 Function Interfaces

The following interfaces basically meet daily usage requirements, including regulator enable, disable, voltage setting, and voltage retrieval:

1. Get regulator

```
struct regulator *regulator_get(struct device *dev, const char *id)
```

Set dev to NULL by default; id corresponds to the regulator-name property in the DTS.

2. Release regulator

```
void regulator_put(struct regulator *regulator)
```

3. Enable regulator

```
int regulator_enable(struct regulator *regulator)
```

4. Disable regulator

```
int regulator_disable(struct regulator *regulator)
```

5. Get regulator voltage

```
int regulator_get_voltage(struct regulator *regulator)
```

6. Set regulator voltage

`int regulator_set_voltage(struct regulator *regulator, int min_uV, int max_uV)`

The caller must ensure that the passed parameters satisfy min_uV = max_uV.

7. Example

```
struct regulator *rdev_logic;

rdev_logic = regulator_get(NULL, "vdd_logic");                  // Get vdd_logic
regulator_enable(rdev_logic);                                                  // Enable
vdd_logic
regulator_set_voltage(rdev_logic, 1100000, 1100000);    // Set voltage to 1.1V
regulator_disable(rdev_logic);                                                 // Disable
vdd_logic
regulator_put(rdev_logic);                                                          //
Release vdd_logic
```

Note: Kernels 4.4 and 4.19 also provide regulator interfaces prefixed with `devm_` to assist developers in managing allocated resources.

3. Debug

3.1 Kernel 3.10

Since the logic underlying drivers related to the PMIC is not complex, the focus lies primarily on the final register configurations. Therefore, the common debugging approach involves directly inspecting RK805 registers via the following node:

`/sys/rk816/rk816_test`

Read register:

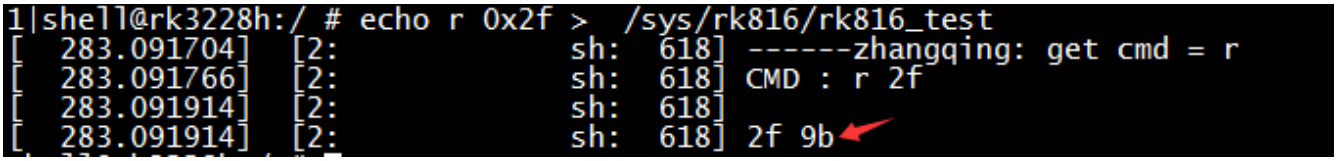
`echo r [addr] > /sys/rk816/rk816_test`

Write register:

`echo w [addr] [value] > /sys/rk816/rk816_test`

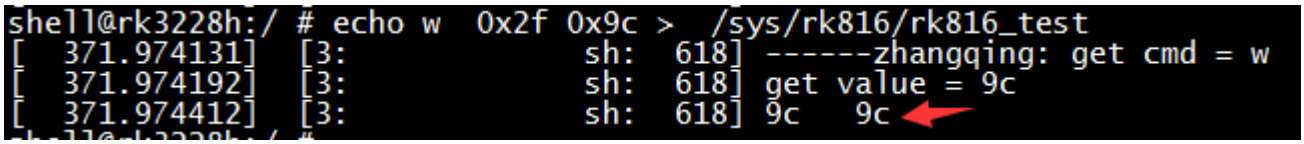
Example:

`echo r 0x2f > /sys/rk816/rk816_test // Read the value of register 0x2f, which is 0x9b`



`echo w 0x2f 0x9c > /sys/rk816/rk816_test // Set the value of Register 0x2f to 0x9c`

It is recommended to perform a read operation after a write operation to verify that the write was successful.



3.2 Kernel 4.4

The command format is identical to that of the 3.10 kernel; only the node path differs. The debug node path on the Kernel 4.4 is:

`/sys/rk8xx/rk8xx_dbg`

3.3 Kernel 4.19

Refer to the commands for the Kernel 4.4.